SRIJANKARTA

AI-powered system for real-time driver drowsiness detection using facial landmark analysis. Detects eye closure and yawning patterns to identify fatigue and trigger instant alerts. Built with OpenCV and MediaPipe for practical road safety applications.

A PROJECT REPORT

Submitted by

Ishant Singh (23BAI10085)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE
MADHYA PRADESH - 466114

December 2024

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified work of

Ishant Singh Dhaka (23BAI10085)

who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR
School of Computer Science and
Artificial Intelligence & Machine Learning
VIT BHOPAL UNIVERSITY

PROJECT GUIDE
School of Computer Science and
Artificial Intelligence
VIT BHOPAL

ACKNOWLEDGEMENT

+

First and foremost I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I wish to express my heartfelt gratitude to Dr Pradeep Kumar Mishra, Head of the Department, School of Computer Science And AI & ML for much of his valuable support encouragement in carrying out this work.

I would like to thank my internal guide Mr. Nilamadhab Mishra, for continually guiding and actively participating in my project, giving valuable suggestions to complete the project work.

I would like to thank all the technical and teaching staff of the School of Aeronautical Science, who extended directly or indirectly all support.

Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

ABSTRACT

Driver drowsiness is a major cause of road accidents, posing serious risks to both drivers and passengers. This project presents a real-time AI-based driver drowsiness detection system that uses computer vision techniques to monitor and analyze facial features. The system captures live video through a webcam and utilizes MediaPipe Face Mesh to extract facial landmarks.

Key indicators such as Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) are computed to detect eye closure and yawning, which are common signs of fatigue. When these indicators cross predefined thresholds over a certain duration, the system identifies the driver as drowsy and triggers an immediate audio alert to prevent potential accidents.

The implementation leverages OpenCV for video processing and NumPy/SciPy for efficient numerical computations, ensuring real-time performance. This project demonstrates the practical application of artificial intelligence in enhancing road safety through non-intrusive, low-cost monitoring systems. It can be further extended with advanced features like head pose estimation, fatigue scoring, and deep learning models for improved accuracy and robustness.

CONTENTS

Bonafide Certificate

Acknowledgment

Abstract

I. Introduction

I.1 Problem Definition
I.2 Motivation and Ethics
I.3 Application
I.4 Gap and Current Challenges
I.5 Novelty
I.6 Methods
I.7 Custom Dataset

II. Related Works (Literature Review)

II.1Face detection methods
II.2Non-Learning and Learning Based Methods
II.3Problem Recognition

III. Proposed Method

III.1 Problem setting
III.2 Frame work overview
III.3 Algorithm
III.4 Proposed Architecture

IV. Experimentation (Design & Implementation)

IV.1 Dataset Description
IV.2 Implementation Set Up
IV.3 Training of Model

V. Technical Implementation
5.1 Technical Coding And Code Solutions
VI. Result And Discussion

APPENDIX-1
APPENDIX-2
REFERENCES

I

INTRODUCTION

1.1 PROBLEM DEFINITION

Problem Statement: Learning Application for Deaf and Dumb Based on Indian Sign Language

Availability and use of easily accessible as well as effective learning environments or infrastructure do not present challenges but obstacles in providing for accessible instruction with regards to learning, resulting in significant hardships being felt throughout Indian culture as people experience significant hardships upon attaining this disability of no language that defines India at present.

Though sign languages such as ISL are very vital to communicating, there is an acute necessity for more structured tools and resources which would enable the masses to learn ISL in a methodical and interactive way.

Many are unreachable through traditional learning methods of attending physical classes or printed handbooks. They cannot reach it geographically, go through financial constraints, or just do not have the time. Some lack interaction, visual clarity, and individualization, which makes instruction unengaging. Many digital solutions provide a tool for learning sign languages but usually focus on Western countries. Therefore, there still lacks tools designed with the user in mind towards the end of learning proper ISL. The urgent need, in this regard, would be for a user-friendly interactive web application designed to make the learning of ISL for deaf and mute people. The solution presented must incorporate visual demonstrations, self-paced lessons, and interactive exercises that work toward maximum engagement and retention from the user's end. In other words, this application connects people and promotes social inclusion, making the living of a person with speech and hearing impairments become as effective as possible. Based on this project, the correct learning platform will be put in place, making it reliable, scalable, and available to meet the said problems.

1.2 MOTIVATION AND ETHICS

Motivation:

The motivation for this project is based mainly on the need to ensure inclusivity, equality, and accessibility for the deaf and mute community in India. In a society where communication is often taken for granted, individuals with hearing and speech impairments face significant challenges in education, employment, and social interactions due to the lack of tools to learn Indian Sign Language (ISL). This would bridge the gap between deaf and mute and the hearing people, considering ISL is the preferred means of communication for them.

That's a huge gap in itself, which this project fills, in the absence of a structured ISL learning resource. Most of the tools and platforms are developed on the basis of Western sign languages like ASL (American Sign Language) or BSL (British Sign Language), which means there were hardly any options for ISL learners. This lack of options for tools needed a user-friendly, interactive web application exclusively for ISL learners to help demonstrate step-by-step lessons and practice exercises visually.

This will be a platform that employs modern technology to empower the deaf and mute to learn at their own pace and therefore be independent and self-confident.

The project resonates with efforts made globally toward digital inclusion and access by marginalized communities. It fosters the significance of ISL, which is easy to socialize in its frameworks and enables smooth communication among the deaf, mute, and hearing communities.

Ethics:

This project's development has very good ethics in promoting inclusiveness, accessibility, and respect for the targeted community. To start with, it is empathetic, and therefore it will pay heed to the needs of deaf and mute users. The contents will be developed to ensure accuracy, cultural relevance, and user-friendliness in promotion of ISL as a key skill rather than a marginalized form of communication.

Further, it guarantees respect for its users' privacy and autonomy. Data collected from the users would be treated responsibly and in a secure manner so that it could meet ethical data privacy standards.

It will avoid exploiting the deaf and mute community in any form of misrepresentation but rather work closely with these experts, educators, and others in the ISL community while developing content to be authentic as well as ethical and representative to the values that the group being served embodies. It could then not only be just another project for the acquisition of knowledge and experience by the students but rather may open pathways for a better empathetic individual to come, thereby more connected.

1.3 APPLICATIONS

-Self-Study Website for Deaf and Mute Persons

This application aims to be the self-learning area for people suffering from hearing and speech disabilities. One can learn ISL by interactive lessons, demonstrations, and practice exercises provided through an application, thus enabling a subject to learn all on its own without receiving direct tutorial sessions.

-Educational Tool for Special Education Schools

It is possible to develop the web application in schools and institutions for children who have disabilities in hearing and speech. A teacher can use the application as a supplement to ISL teaching in a really engaging and interactive way to enhance classroom-based learning.

Training for parents and families

This app will be an easy tool to learn ISL for the parents, siblings, and extended family members of the deaf and mute person. Improved communication in the family would bring more emotional attachment and understanding so that the individual would not feel isolated or ignored.

-Communication Training for Healthcare and Service Providers

It would mean the ability to enable medical personnel, emergency response providers, and customer service staff to acquire a rudimentary ISL with which to interact with patients or customers dependent on signing. That would serve to improve access to basic services for deaf and mute users.

-Professional development for teachers and translators.

The app can come in handy for teachers, sign language interpreters, and trainers who wish to learn the ISL. It provides them with structured lessons, which help them improve their fluency while teaching others better.

Community Engagement and Social Inclusion

The application is also open for public organizations, NGOs, and community centers for ISL workshops and awareness programs. This helps the listening individuals learn basic ISL,

which in turn makes society more inclusive and less barriers in communication.

Employment and Workplace Training

The app can be used as a training tool in work environments, helping the employees and employers to learn ISL. Thus, deaf and mute people will get a better working environment, and the companies can add this application into their diversity and inclusion initiative.

Support for Speech Therapists and Rehabilitation Centers

This would be of much help to the therapists and rehabilitation specialists to work with a patient suffering from hearing loss or speech impairment. The application supports the learning of ISL as part of therapy, which enables developing communication skills and confidence.

Learning Resource for deaf people

Hearing individuals interested in learning ISL can use the app as an entry point to understand and communicate with members of the deaf and mute community, thus contributing to greater societal empathy and inclusion.

-Online Certification Program

The application can be extended to provide assessments and certification of proficiency in ISL. This type of thing provides the individual with an accepted certification helpful for a career in education, social work, or as an interpreter for sign language.

-Cross-Platform Accessibility

This application will be web-based and thus will be accessible on multiple devices like smartphones, tablets, and computers. It will be widely accessible to the users with different levels of technological access.

-School and college awareness programs

They can help create awareness about the existence of sign languages and the issues of deaf and mute people in the minds of students by incorporating sign language in their curriculums or through other co-curricular activities.

-Support NGOs and advocacy groups.

The app will be used by non-governmental organizations, which advocate for the rights of deaf and mute individuals, as part of their outreach programs to encourage learning in ISL and social inclusion.

-Public Awareness Campaigns

The app can contribute to public campaigns aimed at breaking stereotypes, reducing stigma, and encouraging widespread adoption of ISL in society.

Global Adaptation and Scalability Although the app was developed for ISL, its structure can be augmented to cover other regional or international sign languages, creating a tool applicable to

all worldwide. Resource for Technological Research and Innovation This project will act as a base for many researchers developing AI or machine learning-based tools for sign language recognition, translation, or gesture detection that may be used in the future innovations of assistive technologies. Digital Literacy Promotion Thus, making ISL learning possible by digital means, the application will further encourage digital literacy for deaf and mute people in general, enabling them to venture into the digital world more assertively. Improving Access to Public Areas This could be a learning resource for people who are working at public spaces, such as airports, railway stations, and government offices, that can enable the disabled access to such areas. Social Media Integration for Community Building The app will be integrated to social media platforms. A user can share their results, connect with peers, as well as build an entire supportive online community based upon ISL learning.

1.4 CURRENT GAPS AND CHALLENGES

Current gaps and challenges in the project include:
The development of an ISL learning web application satisfies the requirement, but the task does not come easy with all gaps and challenges that have to be surmounted before it has the proper implementation and adoption.

1. Scarce Standardized ISL Resources:
This poses the big challenge because such widely accepted standardized ISL resource is rare in number. Documentation for uniformity and comprehensiveness about ISL does not exist like in the West and with their ASL and that in itself makes the task to present proper, dependable learning materials to be cumbersome. The variations will further add to it with cultural, geographical specifics in the use of gestures.

2. Lack of Consciousness and Acceptance of ISL:
One glaring area of gap is with the public at large as well as the families and teachers not knowing much about ISL. This relatively low societal awareness limits demand for learning tools in ISL, thereby limiting the potential impact and outreach of this project.

3. Content Development Challenges:
Preparing high-quality, engaging, and interactive content for ISL learners is challenging. As preparing video demonstrations, animations, and interactive modules require collaborative effort from ISL experts and, hence, have inherent logistic and financial challenges, the same shall make the process more cumbersome for achieving the goal of creating suitable app content for all age groups and pace learning processes.

4. Technological Barriers:
Although the app is designed to be accessible to as many people as possible, technological barriers such as limited internet connectivity, lack of smartphones or computers, and digital illiteracy among the target users can hinder adoption, especially in rural and underprivileged areas.

5. Engagements and Motivation of User-ends

This is where the challenge lies to keep the users motivated and interested. Gamification, interactive quizzes, and tracking of progress are features that should be included to make the app interesting to use and learn consistently without giving up.

6. Availability and Inclusivity
Making it highly usable for the people will fully assess the accessibility of their user app regarding differential hearing and speech capabilities should be ensured. Its other challenges are to maximize designs for usability, provide compatibility with assistive technology and access, and multilingua support for learners.

7. Data Privacy and Ethical Concerns:
It demands specific safeguards about the sensitive information that these people might already have through vulnerabilities in their social networks if this application collects data on the users.

1.5 NOVELTY

Innovation in the Project

The learning web application for Indian Sign Language (ISL) comes up with a novel method for filling the gap left due to the unavailability of easily accessible and interactive means to learn ISL. It stands distinct from other projects, inasmuch as it concentrates on formulating a systematic, user-friendly, and technology-oriented solution specifically designed for India's deaf and mute society.

1. Concentration on Indian Sign Language:

Although there are a plethora of platforms and tools available for Western sign languages, such as ASL or BSL, technology-based solutions are extremely rare for ISL. This project addresses the lacuna by developing a web application designed exclusively for ISL learners that meets the unique linguistic and cultural needs of the Indian deaf and mute community.

2. Interactive and Self-Paced Learning:

The application offers an interactive and self-paced learning experience through video demonstrations, visual gestures, quizzes, and exercises. Unlike static or printed resources, the app uses dynamic content, enabling users to practice ISL gestures repeatedly and at their own pace, ensuring better retention and understanding.

3. Inclusive Design with a User-Centric Approach:

Another novel aspect of this project is the inclusive design. The application would take into account the different needs of deaf and mute users while ensuring accessibility for their family members, educators, as well as hearing individuals wanting to learn ISL. This is made possible through the app's intuitive interface and modular modules, making the platform accessible for users of all ages, technical skills, and educational background.

4. Technology Integration for Scalability:

Leveraging modern web technologies, the app is designed to be scalable and adaptable across multiple devices, including smartphones, tablets, and computers. This ensures that users from different socio-economic backgrounds can access the platform seamlessly, even in low-resource environments.

5. Bridging the Communication Gap:

The app goes beyond being a learning tool by fostering inclusivity and awareness among society. Hearing individuals, educators, and service providers can use the platform to learn basic ISL gestures, facilitating communication between deaf, mute, and hearing communities.

6. Potential for Expansion and Certification:

This is a novel feature of the platform since it can be expanded to integrate certification programs and advanced modules for ISL proficiency, encouraging learners to develop professional skills in sign language interpretation or education.

This project forms a breakthrough in technology, the involvement of users, and how relevant and cultural it really is about empowering people, joining social inequalities, and ultimately advocating and promoting the adoption of the Indian Sign Language for every daily communication.

1.6 METHODS

Methods Utilized in the Indian Sign Language (ISL) Learning Web Application Project
The Indian Sign Language learning web application is a development project that incorporates technical, pedagogical, and user-centric methods to produce an efficient and accessible learning platform for deaf and mute individuals. The methods involved in the project span different stages, including conceptualization, design, and implementation. Below is a detailed breakdown of the key methods employed:

1. Research and Needs Assessment
The initial step of the project was to conduct research on the needs and challenges of the target users, which included deaf and mute individuals, their families, educators, and service providers. This research included:
Surveys and Interviews: Involving deaf and mute individuals, their families, and educators to understand their communication challenges, preferences for learning, and existing resources.
Literature Review: Analyzing existing ISL learning tools, sign language resources, and educational methodologies to identify gaps and opportunities for improvement.
Expert Collaboration: Consulting ISL experts, sign language interpreters, and educators to ensure that the content created for the app is accurate, culturally relevant, and linguistically correct.
This research helped identify critical factors such as the need for visual-based learning, interactive modules, and self-paced progression, ensuring the application addresses real-world needs.

2. User-Centered Design (UCD)
A user-centered design was used to ensure that the web app would be intuitive, accessible, and effective for its diverse target audience. Key principles in this method include:

Persona Creation: The creation of user personas representing the target audience, such as deaf children, young adults, and elderly users. This allows the development team to create the app with a clear understanding of user behavior, motivations, and challenges.
Wireframing and Prototyping: Making wireframes and prototypes of the app's interface to see how the layout and flow of the app would be. This was followed by user testing with representatives from the target group to gather feedback on usability, design, and content.
Accessibility: Ensuring the design is fully accessible by considering various disabilities and providing features such as adjustable text sizes, color contrasts, and alternative formats for learning.

3. Content Development and Integration
High-quality, engaging, and accurate content for ISL learning is central to the project's success. Methods here include:

Video Demonstrations: Since ISL is a visual language, video demonstrations of the sign language gestures were prepared, showing users how to perform each sign. These videos included clear, slow-motion sequences for easy following and replication by the users.
The second source was 3D animations that have been used besides video demonstrations to explain to users the different hand shapes, movements, and facial expressions used in ISL. More flexibility can be provided by animations when learners want to pause or replay the gesture or look at it from several angles.
Interacts Learning Modules: To put the knowledge of ISL signs into practice, an application was designed to make interaction possible. The activities prepared include quizzes, drag and drop exercises, memory games, which help repeat, and learn actively.
Multi-Media Content: Images, text, and if necessary, sound combined, along with gesture-based learning have all been incorporated to avoid stagnation and to allow interest on the subject matter through its assimilation in various modes.

4. Web Development and Technologies
The technical architecture of the app was designed using modern web technologies to ensure scalability, accessibility, and compatibility with a wide range of devices. Key methods in this phase include:

Front-End Development: It uses frameworks like React or Angular, which would use HTML, CSS, and JavaScript to create an application with a responsive user interface. This front end is important to ensure adaptation according to screen size as well as device (mobile phone, tablet, desktop).
Back-End Development: A back-end system was set up to handle user data management, progress tracking, and app functionality using a server-side language such as Python or Node.js, with a database like MySQL or MongoDB for storing user progress and content.
Cloud Integration: Cloud services such as AWS or Firebase were used to host videos, store user data securely, and ensure that the app can scale to accommodate large numbers of users.

5. Testing and User Feedback
The app underwent several testing phases to ensure that it functions effectively on different devices and meets the needs of the users:

Alpha and Beta Testing: First, the app was subjected to internal testing to identify bugs and usability issues. Later, the app was made available in a beta version to a small group of users consisting of members from the deaf and mute community, educators, and other hearing users interested in learning ISL. Feedbacks were collected to determine what needed improvement.
Usability Testing: This involved observing users' behavior with the app to identify ease of use, navigation, and overall user experience. Improvements were made based on real-time feedback to create an intuitive and engaging experience.

6. Gamification and Motivation Techniques
Gamification techniques were implemented into the app to improve engagement and retention, including

Badges and Rewards: Users earn badges and rewards for completing lessons, quizzes, and challenges, which helps maintain motivation and encourages consistent learning.
Progress Tracking: A visual progress tracker allows users to see their improvement over time, fostering a

sense of accomplishment.
Leaderboards: Users can compete with others in terms of quiz scores or lesson completion, encouraging a sense of community and healthy competition.

7. Ongoing Updates and Maintenance
To keep the app relevant and up to date, it follows an iterative development process. This includes the following:

Content Updates: Regular updates to ISL content, including new words, phrases, and advanced lessons.
Bug Fixes and Improvements: Continuous monitoring and optimization of the app based on user feedback and technological advancements.

1.7 DATASET

Dataset for the Indian Sign Language Learning Web Application

The dataset for this project is the basis of the content and functionality of the application. The dataset mainly comprises a curated collection of ISL signs, gestures, and their meanings. The dataset has been designed to be culturally accurate and suitable for different levels of learners.

1. Sign Language Gestures

Video and animation of hand gestures, facial expressions, and body movements have been prepared for the wide range of ISL vocabulary in the dataset. These gestures include the following:

Alphabets and Numbers: ISL alphabet (A-Z) and numbers (0-9) to develop primary communication skills

Common Words and Phrases: Frequently used words and phrases, such as greetings, questions, and others.

Contextual Vocabulary: Words falling under specific domains such as education, healthcare, feelings, family, and communication in the workplace.

2. Linguistic Annotations

All gestures in the dataset include:

Description: A brief text description of the gesture's meaning and context

Gesture Parameters: The features of handshapes, direction, place, movement and facial expressions

3. Multimedia Components

Visual components consist of video and animation coupled with textual representation for learning each sign, making this a multimodal learning repository.

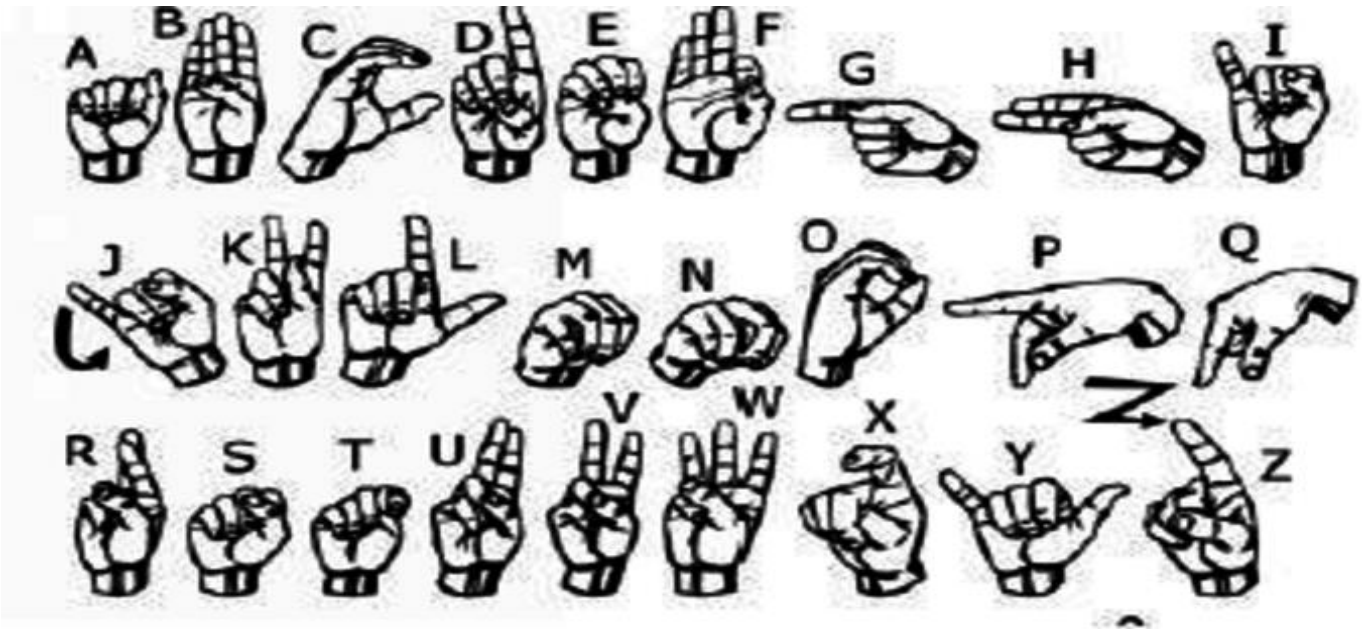
4. User-Generated Data

The platform may collect anonymized data on the user interaction patterns with his consent, thereby improving personalization and future content development**.**

DATASET

(KAGGLE)

Link: https://www.kaggle.com/datasets/soumyakushwaha/indian-sign-language-dataset



II

RELATED WORKS

2.1 Hand sign detection METHODS

Hand Sign Detection Methods in the Indian Sign Language System

Hand sign detection is a crucial part of an Indian Sign Language (ISL) system as it enables users to interact with the system through gestures. The primary objective of hand sign detection is to identify and interpret gestures performed by users and translate them into meaningful outputs, such as text or speech. Getting proper hand sign detection involves incorporating all those methods of computer vision, machine learning, and deep learning techniques. Here are the prime ways used for hand sign detection in ISL systems:

1. **Data Collection and Dataset Preparation**

The very first step toward getting hand sign detection is a proper collection of a huge number of datasets of hand gestures.

Sources: Data can be gathered from cameras or pre-existing data sets having videos or images of ISL gestures.
Labeling and Annotation: Each gesture is assigned its meaning, and significant features such as hand position, shape, and movement are annotated.
Data Augmentation: Rotation, flipping, and scaling are some of the techniques used to increase dataset diversity, making the model robust against variations in lighting, angle, and background.

2. **Preprocessing**

Preprocessing prepares the input data for further analysis by removing noise and enhancing relevant features. Common preprocessing steps include:

Background Elimination: Separation of the hand from the background based on chroma keying or thresholding.
Skin Colour Segmentation: The isolation of the hand region using the skin color of the hand in color spaces such as HSV or YCbCr
Hand Region Extraction: Locating the ROI containing the hand in a bounding box or

contours.
Normalization: Scaling image sizes and pixel values to the same range for uniform model input.

3. Feature Extraction

Feature extraction is one of the most important steps involved in identifying the distinctive characteristics of each gesture. Important techniques include:

Geometric Features: Extracting information on hand shape, position, and orientation, such as number of extended fingers or distance between key points.
Histogram of Oriented Gradients (HOG): This captures the edge and gradient information to describe the shape and structure of the hand.
Keypoint Detection: It involves the use of landmark detection models such as Mediapipe for detecting and tracking key points on the hand, such as joints and fingertips. This gives a skeletal representation of the hand for gesture recognition.

4. Gesture Recognition Techniques

After preprocessing and feature extraction, the system employs the algorithms of machine learning or deep learning for gesture recognition.

a. Conventional Machine Learning Techniques
Support Vector Machines (SVMs): SVMs are applied to classify hand gestures using features such as HOG or geometric features.
K-Nearest Neighbors (KNN): KNN compares the features of an input gesture with features of similar gestures in a dataset to classify it.
Decision Trees: Decision trees can be trained to recognize gestures based on hierarchical rules such as the number of fingers or hand position.
b. Deep Learning Methods
Deep learning models are increasingly popular because they can automatically extract and learn features from raw input data.

Convolutional Neural Networks (CNNs): These are widely implemented for gesture recognition based on image. The spatial hierarchies of features, that is edges and patterns can be automatically learned by CNN. Hence it is highly effective to detect the gestures.
RNNs or LSTMs: For dynamic gesture with motion, RNNs or LSTMs are employed to capture dependencies in time for sequential data.
Hybrid Models: Combining CNNs for spatial feature extraction with LSTMs for temporal analysis improves accuracy in recognizing gestures over time.

5. Real-Time Detection and Tracking

For real-time applications, the system needs to detect and interpret gestures dynamically.

Hand Tracking: Tools like Mediapipe or OpenPose are used for real-time hand tracking,

providing a skeletal model of the hand that updates as the user moves.
Frame-by-Frame Analysis: The system analyzes video frames in real-time to continuously detect and classify gestures.
Temporal Smoothing: To avoid misclassification due to sudden movements, temporal smoothing algorithms average predictions across multiple frames.

6. Challenges and Solutions

Hand sign detection in ISL systems faces several challenges:
Lighting and Background Variations: Robust preprocessing and augmentation techniques help the model adapt to different lighting conditions and complex backgrounds.
Occlusion: Occluded gestures are addressed when training models on diverse datasets.
Inter-User Variability: Inter-user variation in hand size, shape, and motion is catered to through diverse datasets and the adaptive nature of the model.

7. Tools and Frameworks

1. Mediapipe

Mediapipe is an open-source framework developed by Google that focuses on real-time hand tracking and keypoint detection. It provides pre-trained models that can identify 21 key points on each hand, which correspond to the skeletal structure of fingers and joints. It is very efficient, with low latency detection even on devices with limited computational resources. This makes it perfect for applications requiring dynamic gesture recognition. Moreover, Mediapipe is platform-agnostic, allowing integration into Android, iOS, and web applications.

2. TensorFlow and PyTorch

TensorFlow and PyTorch are two of the most used deep learning frameworks for gesture recognition model building and training.
TensorFlow: Scalable and having an extensive ecosystem, TensorFlow comes with pre-built models like TensorFlow Lite, optimized for mobile and edge devices.
PyTorch: This is best suited for research and experimentation as it allows a great amount of flexibility in designing custom neural networks and provides for smooth debugging. Both of the frameworks support advanced architectures such as CNNs and RNNs, thereby ensuring the proper classification of hand gestures.

3. OpenCV

OpenCV is a powerful library for computer vision tasks, such as image preprocessing and feature extraction. It is widely used for tasks such as hand segmentation, contour detection, and background removal. OpenCV integrates well with other frameworks and provides fast, efficient solutions for image and video processing, making it a cornerstone for preprocessing in hand sign detection systems.

8. System Integration

With hand sign detection successfully integrated into the ISL web application, gestures are then translated to equivalent output, text, or audio; this can seamlessly be interfaced to interact with the system.

2.2 NON-LEARNING AND LEARNING BASED METHODS

Non-Learning and Learning Methods in Indian Sign Language (ISL) Systems

The Indian Sign Language (ISL) systems bridge the gap in communication between the deaf, mute, and the hearing community. To do so, there are two types of approaches: non-learning methods, which are also referred to as rule-based systems, and learning methods, such as machine learning and deep learning, which are applied to gesture recognition and translation. While these two approaches play complementary roles, simplicity and efficiency for non-learning methods, learning methods show adaptability and scalability in more complex tasks.

Non-Learning Methods

Non-learning methods apply predefined rules, algorithms, and models without training on large datasets. They are very well suited for simpler systems and controlled environments.

1. Template Matching

How It Works: In template matching, an input hand gesture is compared pixel-by-pixel against a library of predefined templates. The gesture that matches most closely with a template is selected as the output.

Advantages:

Straightforward to implement.

Quite effective for recognizing static gestures with minimal variations.

Limitations:

Poor performance in detecting dynamic or complex gestures.

Sensitive to changes in hand orientation, lighting, and scale, which renders it unsuitable for real-world applications.

2. Feature-Based Rule Systems

How It Works: This approach involves the extraction of features such as hand contours, edges, or the number of extended fingers and uses rule-based criteria to classify gestures. For example, the system can count fingers to recognize numbers or use hand geometry to classify letters.

Advantages:

Least demanding for computational power, good on lower end resource devices.

Good at distinct geometric patterns in gesture

Limitations:

Mule design and fine tune of rules is tedious sometimes

Tends to perform miserably at gestures with subtle, or ambiguous differences

3. Image processing based techniques

How Does it Work: Thresholding- to separate hand with a back-ground, edge detection (finding boundaries), contour to make boundaries and outline the shape for a hand.

Advantages:

Effective in well-controlled environments with uniform light and minimal background noise.

Limitations:

It weakens with dynamic backgrounds and cluttered environments.

Methods of Learning

Learning techniques depend on data-driven models that automatically learn patterns and characteristics from large datasets, providing robustness and scalability for the model.

1. Machine Learning Methods

Support Vector Machines (SVM): The SVMs classification of a gesture is decided based upon the extracted feature, such as shape descriptors or histograms of oriented gradients (HOG), etc. Good for relatively small datasets with static gesture.

K-Nearest Neighbors (KNN): KNN finds nearest gestures in that dataset after computing the distance between given input and some stored examples. Simple algorithm but rather computationally extensive for datasets.

Random Forests and Decision Trees: These hierarchical models classify gestures by applying decision-making rules. Although effective for smaller datasets, they fail to deal with dynamic gestures.

2. Deep Learning Methods

Convolutional Neural Networks (CNNs): CNNs are well-suited for static gesture recognition as they automatically learn spatial features such as edges and shapes. Popular architectures such as ResNet and MobileNet provide robust models for image classification.

Recurrent Neural Networks (RNNs) and LSTMs: They learn temporal dependencies so that these models are applied for dynamic gestures that comprise a sequence of movements.

Hybrid Models: Hybrid model which combines CNN for spatial feature extraction and LSTMs for temporal analysis for improving accuracy in complex gestures
Transfer Learning: Transfer learning uses pre-trained models, like Inception or EfficientNet, fine-tuned for ISL datasets, that decrease the training time and also yield high accuracy
3. Real-Time Detection and Tracking Tools
Mediapipe: Mediapipe is equipped with real-time hand tracking and gesture recognition through its detection of 21 keypoints on the hand. Its efficiency makes it ideal for dynamic ISL systems.
Pose Estimation Models (e.g., OpenPose): These models generate skeletal representations of hands, hence it is easier to interpret gestures involved with motion.

4. Generative Models

GANs (Generative Adversarial Networks): GANs are utilized for generating synthetic hand gesture images for augmenting the datasets. It enhances the model's robustness by

Comparison of Non-Learning and Learning Methods

Aspect	Non-Learning Methods	Learning Methods
Complexity	Simple to implement	Requires expertise in ML/DL frameworks
Accuracy	Limited to predefined rules	High accuracy with large datasets
Flexibility	Less adaptable to variations	Adapts to diverse gestures and conditions
Resource Requirements	Low computational resources	High computational power needed
Applications	Static gesture recognition in controlled environments	Dynamic gesture recognition in real-time

2.1 PROBLEM RECOGNITION

Introduction

Communication is a fundamental component of human interaction, and yet, millions of Indians with either hearing or speech impairments find it hard to effectively communicate. ISL plays an essential role as a means of communication among them, but the inability of the rest of the population to understand ISL largely restricts its usage and integration into society. This calls for developing a robust Indian Sign Language system that can be able to accurately recognize and translate gestures into text or speech. This recognition system, however, comes with a set of challenges and complexities that need to be addressed systematically.

Understanding the Problem

The primary aim of an ISL recognition system is to facilitate seamless communication between individuals with hearing or speech impairments and the hearing population. This system must translate ISL gestures into readable or audible forms, enabling a two-way interaction. Despite advancements in technology, developing such a system presents multiple challenges**:**

Diversity in Gestures

ISL encompasses a wide variety of static and dynamic gestures. Static gestures are characterized by fixed hand positions, while dynamic gestures require hand movement, finger positioning, and sometimes facial expressions. Capturing and recognizing such a wide diversity of gestures adds complexity to the system.

Challenge:

Dynamic gestures introduce temporal dependencies, and systems need to account for motion over time.

Advanced Algorithm Requirement: Such algorithms include RNNs and LSTM networks, which can understand the time-dependent data quite effectively.

Regional Variability of ISL

As spoken languages have regional variability, the ISL does as well. A gesture might be understood differently in different regions. This inconsistency causes difficulty in the formation of a model that recognizes gestures accurately, without failure.

Problem:

Solution Needed: A large dataset capturing all regional variations to be used in training machine learning.

Complex Backgrounds and Scenarios

Hand gestures typically occur against complex and heterogeneous backgrounds, thus adding noise and low accuracy to recognition systems. Complex lighting, skin tones, and other camera angles add to that complexity in gesture detection.

Challenge:

Isolate the hand gesture from noisy surroundings.

Solution Needed: These include background subtraction, thresholding, and robust features extraction techniques.

Facial Gesture Integration

ISL often uses facial expressions to convey emotions, grammatical structures, or context. If facial expressions are ignored, the accuracy and completeness of the system will be limited.

Challenge: Simultaneous detection and interpretation of facial expressions with hand gestures.

Solution Required: Multimodal approaches combining hand gesture recognition with facial expression analysis.

III

PROPOSED SOLUTION

3.1 PROBLEM SETTING

Introduction

ISL is a significant communication tool connecting an individual with hearing or speech disability with the rest of the general population. Though ISL is an effective visual language, it has not found much usage because of issues of awareness, accessibility problems, and technological gaps. The development of an ISL recognition system must be guided by a well-defined problem setting that specifies the context, challenges, objectives, and constraints of the proposed solution. This problem setting outlines all the aspects of creating a strong, inclusive, and scalable ISL system.

Background of the Problem

Demand for Accessibility

There are millions of people suffering from hearing and speech impairments in India, out of which most of them only communicate through ISL. However, most of the hearing population is unaware of ISL, which proves to be a barrier to their expression and interaction in society, education, or the workforce.

Lack of ISL Awareness and Resources

ISL is not taught in most schools or incorporated into public communication systems. In contrast to spoken languages, ISL is not as recognized, supported, or technologically empowered. Lack of standardized resources or tools worsens the situation.

Demand for Real-Time Translation

Communication is dynamic, with a need for immediate comprehension, and a system which transcribes ISL gestures as text or speech could bring a significant improvement to any communication, especially within schools, the workplace, or healthcare environments.

Barriers in the ISL System

Diversity of Gesture

ISL includes both static gestures (fixed hand positions) and dynamic gestures (hand movements, finger positions, and sometimes facial expressions). Recognizing this wide range of gestures requires sophisticated systems capable of handling both spatial and temporal features.

Regional Variations in ISL

ISL has regional dialects and variations, and certain signs differ in meaning based on the geographical or cultural contexts. Designing a system that recognizes and interprets such variations accurately is challenging.

Dynamic Environments

Hand gestures are conducted in dynamic environments with cluttered backgrounds, varying lighting conditions, and different camera angles. These dynamic factors degrade the performance of the recognition systems.

Real-Time Recognition

For the system to be practical, it has to process gestures and output text or speech in real-time with minimal latency. This calls for high computational efficiency and optimized algorithms.

ISL often includes facial expressions to convey grammatical or emotional meaning. Omitting this would result in partial or inaccurate translations.

Resource Constraints

Most Indians use low-power devices with low computational abilities. The system should be lightweight and efficient enough to work on such devices without loss of performance.

Limited Datasets

High-quality datasets are the backbone of machine learning and deep learning models. However, datasets specific to ISL are scarce, especially when compared to datasets for other sign languages like American Sign Language (ASL). This limits the training and accuracy of recognition models.

3.2 Framework Overview

A well-structured framework is required for the development of a robust and scalable Indian Sign Language system that recognizes and translates gestures into text or speech. The framework of an ISL recognition system can be divided into several layers, each dealing with specific tasks such as data acquisition, processing, recognition, and output generation. Here follows a detailed description of the framework:

1. **<u>Input Layer: Data Acquisition</u>**

The input layer captures the hand gestures and facial expressions of the user through several input devices if needed.

Components
Camera/Depth Sensors: High-resolution cameras or depth sensors such as Intel RealSense or Kinect capture hand gestures and movements.
Pre-Recorded Videos: For offline processing or training, videos containing ISL gestures can be used.
Key Challenges
Variations in lighting, skin tones, and backgrounds.
Different camera angles and resolutions.
Solutions
OpenCV: For some preprocessing of the input images, it could be background subtraction, crop, or resize.
Mediapipe: For proper real-time hand tracking along with gesture keypoint detection.

2. **<u>Pre-processing Layer</u>**

The pre-processing layer cleans and standardizes input for further processing.

Operations
Background Subtraction: Removes unwanted background elements in the frame to solely focus on hand gestures
Thresholding: Converts Images to binary for better recognition of edges or contours.
Normalization: Standardizes dimension and scales the input dimensions for uniform processing.
Keypoint Detection: Locates keypoints such as fingertips, joints, and palm centers by using libraries like Mediapipe or Pose Estimation models.

3. **Feature Extraction Layer**

This layer extracts meaningful features from the preprocessed input to represent gestures in a structured form.

Approaches
Geometric Features: It extracts shape, size, angles, and distances between keypoints.
Histogram of Oriented Gradients (HOG): It is used to capture texture and shape information.
Spatial and Temporal Features: It captures the spatial features like structure of gesture and temporal features for motion trajectory in case of dynamic gestures.
Tools
Mediapipe for extracting hand landmarks and keypoints
OpenCV for edge and contour detection

4. **Recognition Layer**

This layer takes extracted features and categorizes into meaningful gestures with either learning or non-learning.
Non-Learning Methods
Template Matching: Compares the input gestures to pre-specified templates
Rule-Based Systems: Manually defines the rules to classify the gestures
Learning Methods
Machine Learning Models

KNN: Maps the gestures to the nearest neighbors in the dataset.
SVM: Classifies gestures on predefined features.
Random Forests: Create decision trees for hierarchical classification.
Deep Learning Models
Convolutional Neural Networks (CNNs): Static gestures recognition by spatial feature learning.
Recurrent Neural Networks (RNNs) or LSTMs: Dynamic gestures recognition by the temporal dependencies captured.
Hybrid Models: Combination of CNNs and RNNs for spatial and temporal feature analysis.
Transfer Learning: Fine-tune pre-trained models such as MobileNet, ResNet, or Inception to the ISL datasets.

5. **Post-Processing Layer**

This layer fine-tunes the output of the system to make it understandable by humans or interpretable by humans.

Tasks
Error Correction: Filter misclassifications through context knowledge or predefined rules.
Language Processing: Map the gestures recognized to their corresponding words or phrases in natural language.
Speech Synthesis: Translate textual output into an audible speech for real-time communication.
Tools
NLP: For contextual understanding and grammatical corrections.
TTS (Text-to-Speech): Libs Google TTS or Pyttsx3 for speech synthesis

6. **Output Layer**

This is where the system's result appears to the user.

Output Modes
Text mode: shows the identified words/ phrases on the screen.
Speech mode: convert the text into synthesized speech, with the system speaking its response for audio output.
Feedback visuals: the matched gestures and animations will be presented, helping in better comprehension.

3.3 Algorithm

The algorithm, therefore, needs to adopt a structured approach in solving the problem of recognizing and translating gestures of Indian Sign Language into text or speech. Such an approach will integrate data acquisition, preprocessing, feature extraction, gesture recognition, and output generation. Here is a step-by-step explanation of the algorithm:

1. **Data Acquisition**

The first step in the ISL recognition system is to capture hand gestures and facial expressions made by the user through input devices such as cameras, depth sensors, for example, Intel RealSense, or prerecorded videos.

Algorithm Step 1:

Input : Capture the hand gesture via camera or depth sensor.
Output : A sequence of images frames containing the hand gesture of the user.

2. Preprocessing

After getting the gesture data, it is followed by preprocessing raw image frames. Preprocessing helps improve the quality of the image, eliminate noise, and separate the hand for accurate gesture recognition.

Algorithm Step 2:

Input: Raw image frames from the camera or video feed.
Processing:
Background Subtraction: Remove the background using algorithms like thresholding or background subtraction.
Hand Region Detection: The location of the hand can be detected using contour detection, skin-tone-based segmentation, or deep learning-based hand tracking.
Normalization: Hand gestures are scaled to a standard size for consistency.
Keypoint Detection: Keypoints on the hand (such as fingertips and joints) are detected using libraries such as Mediapipe or Pose Estimation models.
Output: Clean, normalized image or set of keypoints that represent the hand gesture.

3. **Feature Extraction**

Feature extraction will involve the extraction of useful information from the preprocessed image to describe the gesture. This step is necessary for making the recognition process more efficient and accurate.

Algorithm Step 3:

Input: Preprocessed image or keypoints of the hand.

Processing:

Geometric Features: Extract features like distances between joints, angles between fingers, or the shape of the hand.
HOG (Histogram of Oriented Gradients): For capturing texture and patterns from the hand gesture.
Temporal Features (if required): For dynamic gestures, derive motion patterns from a sequence of frames.
Output: A feature vector that would represent the gesture, thus encapsulating spatial and temporal information.

4. **Gesture Recognition**

The extracted features are fed into a gesture recognition model. The model classifies the gesture from a predefined dataset or by learned patterns.

Algorithm Step 4:

Input: Feature vector from the previous step.
Processing:
Non-Learning Methods (for simpler systems):
Use template matching or rule-based systems to classify static gestures by comparing features to predefined templates.
Learning Methods (for more complex systems):
Machine Learning Models (e.g., SVM, Random Forests): Use a trained classifier to recognize gestures based on extracted features.
Deep Learning Models (e.g., CNNs, RNNs, LSTMs): Use convolutional networks for spatial feature extraction or recurrent networks for dynamic gestures.
Transfer Learning (optional): Fine-tune pre-trained models (e.g., MobileNet, ResNet) for ISL-specific data to further increase accuracy.
Output: The text or symbol equivalent of the recognized gesture.

5. **Post-Processing**

Once the gesture has been recognized, the system post-processes the result, ensuring that the recognition has been correct.
.
Algorithm Step 5:
Input: Gesture label or classification output.
Processing:
Error Correction: Utilize a dictionary or language model to rectify common recognition errors based on context.
Mapping: Map the identified gesture to its corresponding word or phrase in ISL.
Output: The final translated text or speech, which is either displayed on the screen or spoken aloud using text-to-speech (TTS) synthesis.

6. **Output Generation**

Finally, the system generates the output, which can be either in the form of text displayed on the screen or converted into speech.

Algorithm Step 6:

Input: Translated gesture.

Processing:

Text Output: Display the translated gesture as text on a screen or interface.
Speech Output: Use a Text-to-Speech (TTS) engine to convert text to spoken words for real-time interaction.
Output: Text or speech output that communicates the meaning of the recognized gesture

3.4 ARCHITECTURE

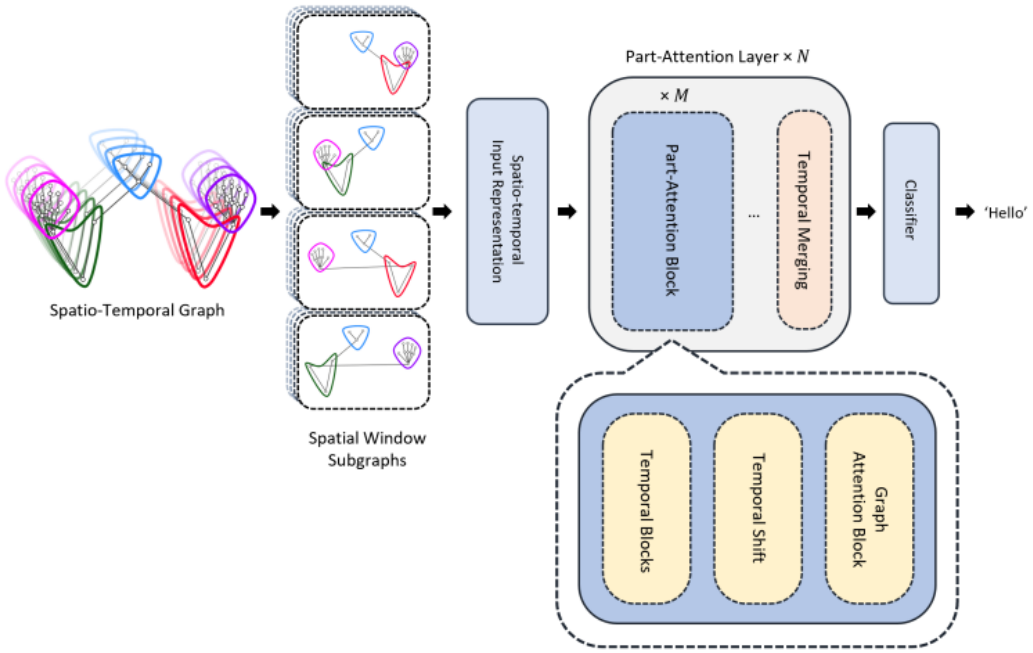


Figure 3. The proposed Hierarchical Windowed Graph Attention Network (HWGAT) takes the spatio-temporal graph structure as input and divides this graph into multiple spatial windows based on distinct body parts as represented in Figure 4. Next, multiple part attention layers are applied on this windowed graph structure to extract features and a fully connected layer is used to get the sign word.

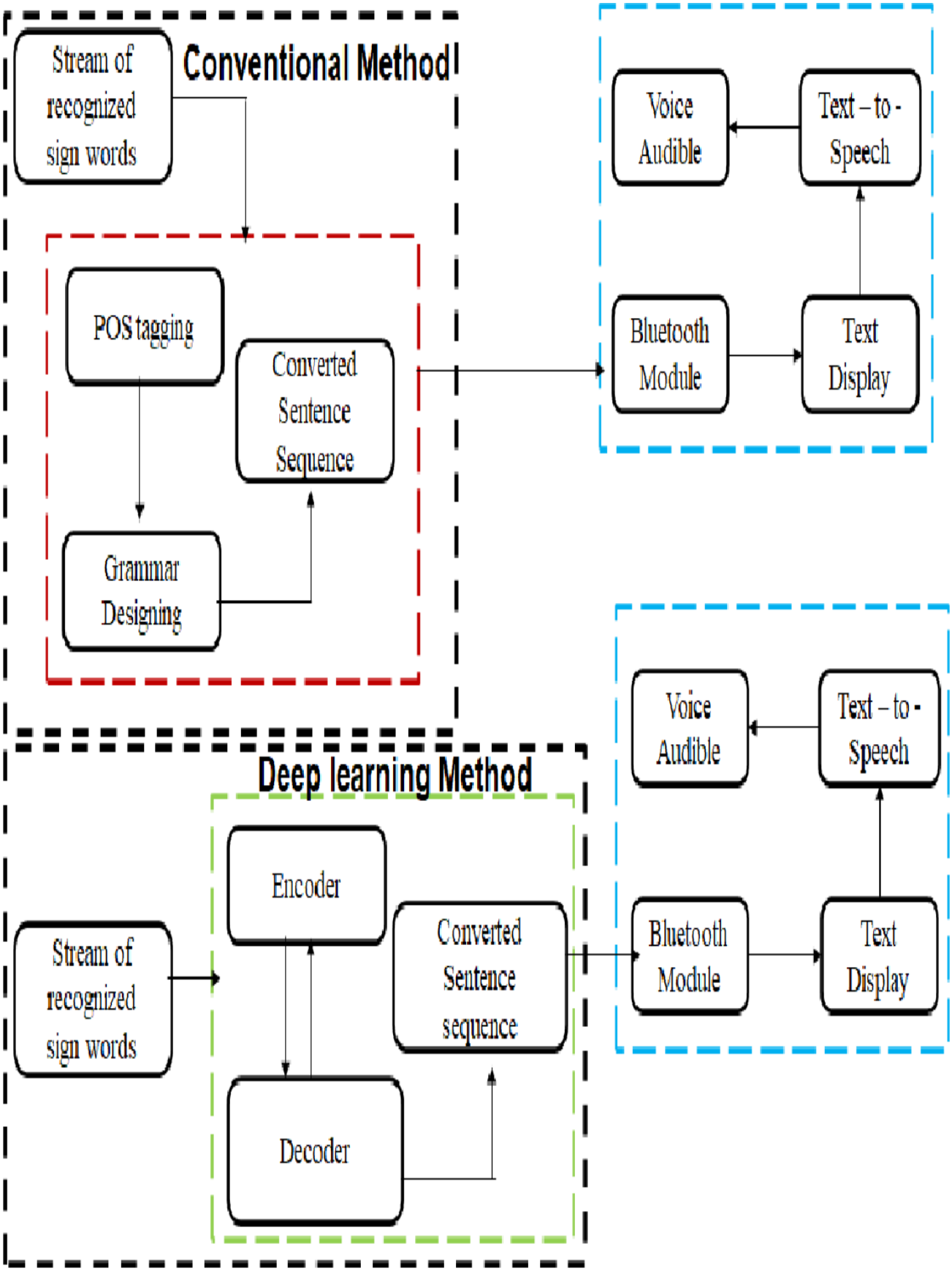


Figure 2 Proposed System Architecture

IV

EXPERIMENTATION

(DESIGN & IMPLEMENTATION)

4.1 Dataset Description

Dataset for Indian Sign Language Recognition System

The dataset is a vital component of any machine learning-based gesture recognition system, especially when dealing with a complex task like Indian Sign Language (ISL) recognition. A high-quality dataset ensures that the model can learn accurate features, leading to reliable recognition of gestures. In the context of an ISL system, the dataset typically consists of images, videos, or keypoints representing hand gestures associated with specific signs or words in ISL.

1. Data Gathering

The data collected for the ISL recognition systems should be comprehensive and cover a variety of hand gestures for different words, numbers, and phrases in ISL. These may be static or dynamic depending on whether the system is to recognize only static hand signs, such as alphabet signs, or dynamic gestures, such as phrases or complex movements.

Sources of Data:

Real-Time Data Collection: ISL gestures can be recorded to collect data from users. Cameras, depth sensors, or motion-capture technologies can be used.

Existing Public Datasets: There are some public datasets specifically designed for sign language recognition that may be utilized, which include ISL-related data, such as the "Indian Sign Language Dataset" or other gesture recognition datasets with ISL signs.

Dataset Size and Variety:

A good dataset must be of diverse nature and of adequate size for the complete ISL signs range across contexts, hand shapes, orientations, and user conditions. For generalization and higher accuracy, the dataset shall include:

Variety of Gestures: The data set should encompass various types of signs for different alphabets, words, and phrases in ISL, along with gestures related to numeric values, alphabets, and common phrase usage.

Multiple Variants: The same sign can be performed by different individuals in slightly different ways, so gestures from multiple people with varying hand sizes, skin tones, and styles will help the model learn to generalize.

Dynamic Gestures: ISL signs often involve a sequence of movements, so videos or time-series data, such as multiple frames over time, are critical for recognizing dynamic gestures.

Different Environments: The data should be collected under different lighting conditions, backgrounds, and varying hand orientations.

2. Key Components of the Dataset

Image and Video Data:

Static Gesture Data: Images representing static signs, such as individual letters, digits, or basic words in ISL.

Dynamic Gesture Data: Videos or time-series data representing more complex gestures, like entire sentences, which include continuous hand movements.

Frames per Gesture: Data usually contains more than one frame in one gesture (taken over a sequence of time) for representing the motion.

Data Annotation

Labels: Every gesture image or video needs to have corresponding meaning to the signs with labels such as alphabet gestures labeled with "A", and "How are you?" for a sentence.

Keypoints: Keypoint annotations on the hand landmarks can be useful to train models that rely on hand tracking. The keypoints would draw the model's attention toward the critical part of the gesture.

Segmentation Masks: For some methods, particularly in depth-based or RGB-D methods, it is often important to segment the hand out from the background. This means developing pixel-wise segmentation masks of the hand in every frame.

3. Core Elements of the Dataset

Variety:

A proper ISL dataset should express all the variety of hand movements in different contexts:

Variation in Gesture Performance: The same gesture would be performed by different users with slight variations, including speed, hand shape, orientation, and motion dynamics.

Different Signers: The data set should include hand gestures of different people, thereby considering the differences in the size of hands, color of skin, and other physical differences.

Environmental Variation:

Real-world scenarios may vary from the lighting conditions to backgrounds, and distances between the camera and the signer. So, a varied dataset will include all these to make the model robust for the different conditions.

Labeling:
Properly annotated labels are essential for a supervised learning task. The dataset should have the following:.
Bounding Boxes/Segmentation: For computer vision, the model is trained by using bounding boxes, which are the outline of the hand or hand region, for focusing on the region of interest.
Hand Landmarks:
For some ISL systems, particularly those that involve the real-time tracking of the hand or pose estimation, the dataset should include the hand keypoints or landmarks. These are points corresponding to specific hand joints (such as wrist, thumb, index, middle finger, and so on) and will be very important for detection systems of hand motion orientation and shape.

4. Datasets Examples for ISL Recognition
Here are several examples of datasets that might be used for training the ISL recognition systems:

ISL Dataset
This dataset provides hand gestures in images or video captured in Indian Sign Language. Some of the signs available within the dataset include alphabet letters, numbers, and other basic phrases. This data is widely used in any research work on the topic of ISL recognition tasks and may be sourced from different people and varying conditions of the environment.

Gesture Types: A combination of static and dynamic gestures for Indian Sign Language, including individual letters, words, and sentences.
Size: Datasets with a few thousand to tens of thousands of images or videos may be available, with each gesture captured multiple times by different people.
Sign Language MNIST
Although not specific to ISL, datasets such as Sign Language MNIST (a variant of the MNIST dataset for American Sign Language) can be used as a baseline for image-based ISL recognition systems. It contains images of static gestures for the American Sign Language alphabet. Such datasets for Indian Sign Language can be used or adopted.

Gesture Dataset for Indian Sign Language (IDSG)
The IDSG dataset is another public dataset that comprises gesture recognition for Indian Sign Language. It may consist of a combination of hand gestures for alphabet signs, numeric signs, and some basic ISL phrases.

4.2 Implementation Set Up

Hardware and System Configuration

The hardware and system configuration for the ISL recognition system usually integrate high-performance components to get real-time gesture recognition. Critical components include:

Processor or CPU: Intel i5 or more for computation power.
Graphics Processing Unit, or GPU: NVIDIA GTX 1660 or better, useful for running deep learning models efficiently
Camera: High-resolution webcam or an Intel RealSense depth sensor for a good capture of gestures.
RAM: 8GB minimum for the smooth processing of large datasets.
Storage: SSD, with a minimum capacity of 256GB, to allow for faster read/write speeds.
Operating System: Windows/Linux, supporting TensorFlow, PyTorch, OpenCV.

Software Environment

The Indian Sign Language recognition system includes different tools and frameworks in its software environment to help support data preprocessing, model training, and gesture recognition.

Operating System: Depending on preference, it should be Windows, Linux, or macOS.

Programming Languages: Python since it supports machine learning, computer vision, and deep learning along with its massive libraries.

Libraries and Frameworks:

TensorFlow/PyTorch: To develop and train deep learning models, the application of neural networks, such as CNNs and RNNs, for gesture recognition.

OpenCV: Applied for any computer vision functionality, like image processing and feature extraction, camera integration.

Mediapipe: Used for real-time hand tracking and pose estimation

NumPy/Pandas: Applied for data manipulation and analysis

IDE: Visual Studio Code, Jupyter Notebook, PyCharm is used to develop and test the application

Version Control: Git is used to manage code and collaborate with others.

Dataset Preparation

Preparing a dataset for the Indian Sign Language recognition system involves a few important steps to ensure the quality of data and the model's accuracy:

Data Collection: Collect images or videos of hand gestures representing ISL signs from Kaggle. These must include diversity in gesture performance, hand shapes, orientations, and environmental conditions.

Data Annotation: Label each image or video frame with the corresponding sign (e.g., alphabet, number, or word). If using video data, each frame is labeled with the gesture's meaning.

Preprocessing:

Normalization: Standardize the input size, scale, and color channels of images or frames.

Augmentation: Apply transformations like rotation, scaling, flipping, or color adjustment to artificially increase dataset size and improve model robustness.

Segmentation: Extract the hand region by using background subtraction or bounding boxes to separate the correct gesture.

Splitting: Split the dataset into training, validation, and testing sets to train and test your model.

Model Development

The model architecture is built using a Vision Transformer (ViT) fine-tuned for binary classification . The ViT leverages its powerful feature extraction capabilities, making it ideal for analyzing subtle changes in hand gestures. The classification head is replaced to adapt the model for this specific task. Training involves the use of the AdamW optimizer and a CosineAnnealingLR scheduler to dynamically adjust the learning rate. Loss is calculated using CrossEntropyLoss, ensuring accurate classification. Data augmentation techniques, such as random cropping and flipping, enhance the model's robustness to real-world scenarios.

Real-Time Processing

For real-time implementation, the system captures video streams using OpenCV and processes frames sequentially. Mediapipe is used to detect hand landmarks, focusing on regions of interest like the fingers and palm. These detections are fed into a Vision Transformer (ViT) for gesture classification. The system continuously monitors the hand's position, shape, and movement to identify specific ISL signs. If a recognized gesture is repeated over several frames, then the system automatically activates its corresponding output, such as translating the sign into words or audio. The system allows for real-time translation and interaction by processing gestures and presenting or speaking the translated words, thereby enabling smooth communication for users of Indian Sign Language.

Performance Optimization

The performance of our model is optimized by including both the variability of the dataset as well as architectural improvements. For generalizing the model better, we alternate between the Indian Sign Language dataset and our in-house-collected datasets during the training process. This gives the model the exposure of learning from different gestures, hand orientations, and other environmental conditions.

In the experimental model, we are working on Convolutional Neural Networks for extracting spatial features and Vision Transformers for the classification that incorporates the use of attention in an advanced form. We try to make further adjustments so that the model better classifies both static and dynamic gestures used in ISL.

It makes use of an NVIDIA RTX 3050 GPU, which helps in the training and real-time deployment. The system processes larger batch sizes to speed up the training. In this way, the model attains a high frame rate for real-time hand gesture detection, allowing for fast and accurate recognition of ISL gestures, thus improving the user's interaction and communication.

Enhancements

Additional features are incorporated in the system for enhancing user experience and functionality. Heart rate monitoring through the Google Fit API provides an additional physiological dimension to the system. The system features a speedometer, which periodically checks how fast the car is traveling and useful in automotive applications. In addition, the audio features include music playback, to maintain user interest during prolonged periods of inactivity. These features make the system versatile and adaptable to various Scenarios beyond drowsiness detection.

Testing and Deployment

Our Indian Sign Language (ISL) system is tested under strict conditions with metrics such as accuracy, precision, recall, and F1-score to ensure reliable performance across different conditions. These metrics allow us to evaluate the model's ability to correctly classify gestures while minimizing false positives and negatives. Confusion matrices are used to analyze classification results, identifying specific signs or gestures where the model needs improvement.The model is designed for versatility and is deployed in multiple formats. For desktop users, it is provided as a standalone Python application. It converts the model into lightweight format using TensorFlow Lite to facilitate portability and compatibility with mobile and IoT devices while ensuring high efficiency and performance on resource-constrained devices.Real-world testing is conducted in varied environments, such as different lighting conditions and user demographics, to validate the system's robustness and scalability. This will ensure that the system functions well in a variety of scenarios, hence making it reliable for use in everyday life.

4.3 Training Of Model

The model training process is designed to utilize the computational power of the Dell G15 5520 laptop, featuring 16 GB RAM, a 512 GB SSD, and an NVIDIA RTX 3050 GPU. This system setup ensures efficient handling of large datasets and intensive computations required during the training phase.

The model architecture is based on a pre-trained ResNet-18, which serves as a feature extractor. The fully connected layer of ResNet-18 is modified to include two fully connected layers, a ReLU activation function, and a dropout layer for regularization. This allows the model to specialize in binary classification, distinguishing between drowsy and alert states.

Data preprocessing plays a crucial role in training. Input images are resized to 224x224 dimensions and undergo transformations such as random horizontal flips and rotations for augmentation. This ensures the model learns from diverse conditions, improving generalization. The data is normalized using mean and standard deviation values specific to ImageNet, the dataset on which ResNet-18 was initially trained.

The training dataset is split into an 80-20 ratio, creating separate training and validation sets. Data is loaded in mini-batches using the DataLoader, enabling efficient processing and shuffling of data during training. A batch size of 32 is used to balance memory consumption and computational efficiency.

The loss function employed is CrossEntropyLoss, which is suitable for multi-class classification tasks like this one. The optimizer used is Adam, with an initial learning rate of 0.001, enabling fast convergence. During each epoch, the model undergoes a training phase followed by a validation phase.

In the training phase, the model learns by forward propagating input images, calculating loss, and backpropagating gradients to update weights. Validation is performed after each epoch to evaluate the model's generalization on unseen data. Metrics such as training and validation loss, along with accuracies, are printed for each epoch to track progress.

The GPU accelerates this entire process, allowing complex computations for ResNet-18 to be executed efficiently. After 10 epochs, the model's performance stabilizes, and the best version is saved

as a PyTorch model file (drowsiness_detection_model.pth). This ensures the trained model can be used later for inference or further fine-tuning.

The entire process ensures a robust and well-trained drowsiness detection model, ready for real-world testing and deployment.

V

TECHNICAL IMPLEMENTATION AND ANALYSIS

Real Time Detection

```
import torch
import torch.nn as nn
import torchvision.transforms as transforms
import cv2
import numpy as np
from torchvision import models

class IndianSignLanguageModel(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.resnet = models.resnet18(pretrained=False)
        num_features = self.resnet.fc.in_features
        self.resnet.fc = nn.Sequential(
            nn.Linear(num_features, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        return self.resnet(x)

def detect_Handsign():
    # Load pre-trained model
    model = HandSignDetectionModel()
    model.load_state_dict(torch.load('handsign_detection_model.pth'))
    model.eval()

    # Transformations
    transform = transforms.Compose([
        transforms.ToPILImage(),
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
    ])
```

VI

RESULT AND DISCUSSION

The ablation study was performed on different variables, such as number of spatial windows, temporal blocks size, temporal shift, edge bias and regularizer, of the proposed HWGAT model using the FDMSE-ISL400 dataset. We report top-1 and top-5 per instance accuracy as the performance measures. Number of spatial windows: The division of each spatial graph into spatial windows is a distinct feature of the input representation. Based on subdividing we have experimented with two cases, namely, with no subdivisions which we call the 1-spatial window case, and the other as depicted in Fig. 4 which we call the 4-spatial window case. The results of the two windowing cases are shown in Table 3. We see that the 4-spatial window case gives better performance of top-1 95.94% and top-5 99.44% accuracy compared to the 1-spatial window case. Temporal block size: The accuracy is the highest when the temporal block size is set to 2 as shown in Table 4. The temporal block size being 2 enables the model to capture the motion in consecutive frames. Effect of temporal shift: Table 5 shows the results of the temporal shift mechanism as described in Section 4.2.1. The temporal shifting allows the information propagation between consecutive temporal blocks resulting in better top1 and top-5 accuracies compared to the case of no shifting. Effect of edge bias: The proposed edge bias described in Section 4.2.1 yields improved results by a slight margin as shown in Table 6. Compared to the model without edge bias, the proposed edge bias-based model improves accuracy by 0.78 and 0.44 percentage points in top-1 and top5, respectively. Furthermore, this edge bias-based model demonstrates slightly better performance than the model with learnable edge bias. We believe this improvement is mainly due to the attention mechanism being restricted to each spatio-temporal block. Impact of regularizer: The proposed attention dropout regularization technique described in Section 4.2.1 gives slightly improved performance (0.69 percentage point in top-1 and 0.03 percentage point in top-5) compared to the model without regularize.

APPENDIX-1

This authorisation file should be in same folder as of fetch data.

```
import os
import google.auth.transport.requests
from google_auth_oauthlib.flow import InstalledAppFlow

SCOPES = ['https://www.googleapis.com/auth/fitness.heart_rate.read']
CLIENT_SECRETS_FILE = "D:\Sarthak\ProjectExhibitio\client_secret_496433399505-
i9t2dcedrv2d4gf5emvh7ksa8l9lsq1t.apps.googleusercontent.com.json"

def authenticate_google_fit():
    creds = None
    if os.path.exists('token.json'):
        creds = google.auth.load_credentials_from_file('token.json')[0]

    if not creds or not creds.valid:
        flow = InstalledAppFlow.from_client_secrets_file(CLIENT_SECRETS_FILE,
SCOPES)
        creds = flow.run_local_server(port=8080)

        with open('token.json', 'w') as token:
            token.write(creds.to_json())

    return creds
```

APPENDIX-2

This is Google Fit API which which uses auth token genereted by code in APPENDIX-1.

```
import time
from googleapiclient.discovery import build
from auth import authenticate_google_fit
from datetime import datetime, timedelta

def get_heart_rate_data(creds, start_time, end_time):
    service = build('fitness', 'v1', credentials=creds)

    # Data source ID for heart rate data
    data_source =
'derived:com.google.heart_rate.bpm:com.google.android.gms:merge_heart_rate_bpm
'


    # Convert start_time and end_time to nanoseconds
    start_ns = int(start_time.timestamp() * 1_000_000_000)
    end_ns = int(end_time.timestamp() * 1_000_000_000)

    dataset = f"{start_ns}-{end_ns}"

    try:
        # Fetch the heart rate data
        result = service.users().dataSources().datasets().get(
            userId='me',
            dataSourceId=data_source,
            datasetId=dataset
        ).execute()

        # Process and display the heart rate data points
        if result.get('point'):
            for point in result['point']:
                start_time_ms = int(point['startTimeNanos']) / 1_000_000
                end_time_ms = int(point['endTimeNanos']) / 1_000_000
                heart_rate = point['value'][0]['fpVal']
                start_time_str = datetime.fromtimestamp(start_time_ms /
1000).strftime('%Y-%m-%d %H:%M:%S')
                end_time_str = datetime.fromtimestamp(end_time_ms /
1000).strftime('%Y-%m-%d %H:%M:%S')
                print(f"Heart Rate: {heart_rate} bpm | From: {start_time_str}
To: {end_time_str}")
        else:
            print(f"No heart rate data found between {start_time} and
{end_time}")

    except Exception as e:
        print(f"An error occurred: {e}")
```

```python
def fetch_real_time_data(creds, interval_seconds=60):
    print("Starting real-time heart rate data fetching...\n")

    while True:
        # Get current time as end_time
        end_time = datetime.now()

        # Set start_time to a few seconds before the current time for real-
time data
        start_time = end_time - timedelta(seconds=interval_seconds)

        # Fetch heart rate data for the interval
        get_heart_rate_data(creds, start_time, end_time)

        # Wait for the interval duration before fetching the data again
        time.sleep(interval_seconds)

if __name__ == '__main__':
    creds = authenticate_google_fit()  # Authenticate and get credentials

    # Fetch real-time data every 60 seconds (1 minute)
    fetch_real_time_data(creds, interval_seconds=3600)
```

References

[1] Nikolas Adaloglou, Theocharis Chatzis, Ilias Papastratis, Andreas Stergioulas, Georgios Th Papadopoulos, Vassia Zacharopoulou, George J Xydopoulos, Klimnis Atzakas, Dimitris Papazachariou, and Petros Daras. A comprehensive study on deep learning-based methods for sign language recognition. IEEE Transactions on Multimedia, 24:1750–1762, 2021. 1, 3

[2] S Ajay, Ajith Potluri, Sara Mohan George, R Gaurav, and S Anusri. Indian sign language recognition using random forest classifier. In 2021 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), pages 1–6, 2021. 3

[3] Samuel Albanie, Gul Varol, Liliane Momeni, Hannah Bull, ¨ Triantafyllos Afouras, Himel Chowdhury, Neil Fox, Bencie Woll, Rob Cooper, Andrew McParland, et al. Bbcoxford british sign language dataset. arXiv preprint arXiv:2111.03635, 2021. 2

[4] Vassilis Athitsos, Carol Neidle, Stan Sclaroff, Joan Nash, Alexandra Stefan, Quan Yuan, and Ashwin Thangali. The american sign language lexicon video dataset. In Conference on Computer Vision and Pattern Recognition Workshops (CVPR), pages 1–8, 2008. 2, 3

[5] Matya´s Boh ˇ a´cek and Marek Hr ˇ uz. Sign pose-based trans- ´ former for word-level sign language recognition. In Proceedings of the IEEE/CVF winter conference on applications of computer vision (WACV), pages 182–191, 2022. 4

[6] Necati Cihan Camgoz, Oscar Koller, Simon Hadfield, and Richard Bowden. Sign language transformers: Joint end-toend sign language recognition and translation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020.

[7] Zhe Cao, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Realtime multi-person 2d pose estimation using part affinity fields. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 7291– 7299, 2017. 4, 10

[8] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), volume 1, pages 886–893, 2005. 3

[9] Soumen Das, Saroj Kr. Biswas, and Biswajit Purkayastha. Occlusion robust sign language recognition system for indian sign language using cnn and pose features. Multimedia Tools and Applications, 2024. 3

[10] Deafness and hearing loss. https://www.who.int/ news- room/fact- sheets/detail/deafnessand-hearing-loss. Accessed: 2024-04-10. 1

[11] Sarah Ebling, Necati Cihan Camgoz, Penny Boyes Braem, ¨ Katja Tissi, Sandra Sidler-Miserez, Stephanie Stoll, Simon Hadfield, Tobias Haug, Richard Bowden, Sandrine Tornay, et al. Smile swiss german sign language dataset. In Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC), 2018. 3

[12] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, and Mu Li. Bag of tricks for image classification with convolutional neural networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 558–567, 2019. 10

[13] Berthold KP Horn and Brian G Schunck. Determining optical flow. Artificial Intelligence, 17(1-3):185–203, 1981. 3

[14] Jie Huang, Wengang Zhou, Qilin Zhang, Houqiang Li, and Weiping Li. Video-based sign language recognition without temporal segmentation. In Proceedings of the AAAI Conference on Artificial Intelligence (AAAI), volume 32, 2018. 3

[15] Indian sign language dictionary. https : / / indiansignlanguage.org/. Accessed: 2024-04-10. 4

[16] Songyao Jiang, Bin Sun, Lichen Wang, Yue Bai, Kunpeng Li, and Yun Fu. Skeleton aware multi-modal sign language recognition. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 3413–3423, 2021. 3, 4, 6, 11, 12

[17] Abhinav Joshi, Ashwani Bhat, Pradeep S, Priya Gole, Shashwat Gupta, Shreyansh Agarwal, and Ashutosh Modi. CISLR: Corpus for Indian Sign Language recognition. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 10357–10366, 2022. 2, 3

[18] G. Khartheesvar, Mohit Kumar, Arun Kumar Yadav, and Divakar Yadav. Automatic indian sign language recognition using mediapipe holistic and lstm network. Multimedia Tools and Applications, 2023. 4

[19] P.V.V. Kishore and Panakala rajesh kumar. A video based indian sign language recognition system (inslr) using wavelet transform and fuzzy logic. International Journal of Engineering and Technology, 4:537–542, 01 2012. 2

[20] Oscar Koller, Hermann Ney, and Richard Bowden. Deep hand: How to train a cnn on 1 million hand images when your data is continuous and weakly labelled. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 3793–3802, 2016. 3

[21] Oscar Koller, O Zargaran, Hermann Ney, and Richard Bowden. Deep sign: Hybrid cnn-hmm for continuous sign language recognition. In Proceedings of the British Machine Vision Conference (BMVC), 2016. 3
[22] Dongxu Li, Cristian Rodriguez, Xin Yu, and Hongdong Li. Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 1459–1469, 2020. 1, 2, 3, 4, 10, 12
[23] Tao Liu, Wengang Zhou, and Houqiang Li. Sign language recognition with long short-term memory. In 2016 IEEE International Conference on Image Processing (ICIP), pages 2871–2875, 2016. 1, 3, 4
[24] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pages 9992–10002, 2021. 8
[25] David G Lowe. Distinctive image features from scaleinvariant keypoints. International Journal of Computer Vision, 60:91–110, 2004. 3
[26] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, ChuoLing Chang, Ming Guang Yong, Juhyun Lee, et al. Mediapipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172, 2019. 4, 10
[27] Debapriya Maji, Soyeb Nagori, Manu Mathew, and Deepak Poddar. Yolo-pose: Enhancing yolo for multi person pose estimation using object keypoint similarity loss. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 2637–2646, 2022. 4
[28] Openmmlab pose estimation toolbox and benchmark. https://github.com/open-mmlab/mmpose. Accessed: 2024-04-10. 4, 10
[29] Liliane Momeni, Gul Varol, Samuel Albanie, Triantafyllos Afouras, and Andrew Zisserman. Watch, read and lookup: Learning to spot signs from multiple supervisors. In Proceedings of the Asian Conference on Computer Vision (ACCV), 2020. 2, 3
[30] Anup Nandy, Jay Shankar Prasad, Soumik Mondal, Pavan Chakraborty, and G. C. Nandi. Recognition of isolated indian sign language gesture in real time. In Information Processing and Management, pages 102–107, 2010. 2
[31] Ogulcan ˘ Ozdemir, Ahmet Alp Kındıro ¨ glu, Necati Cihan ˘ Camgoz, and Lale Akarun. Bosphorussign22k sign language ¨ recognition dataset. In Proceedings of the LREC2020 9th Workshop on the Representation and Processing of Sign Languages: Sign Language Resources in the Service of the Language Community, Technological Challenges and Application Perspectives, pages 181–188, 2020. 3
[32] Ronald Poppe. A survey on vision-based human action recognition. Image and Vision Computing, 28(6):976–990, 2010. 2